Draft B

Hydrofrac

(*Hydraulic fracturing for distributed protocols, or lineage-driven fault injection for Hydro*)

Hydrofrac is a tool for finding out whether a Hydro program keeps the promise made by its `EventualConsistency` label when messages are lost and nodes crash. It runs the program in the Hydro simulator, records why each run produced the outputs it did, and uses that record to decide which faults to try next. It ends in one of two ways: a concrete set of faults, within bounds the test declares, under which the cluster's members disagree, together with the schedule that shows it; or a certificate that no fault set within those bounds can make them disagree.

The search is Molly's (Alvaro, Rosen, Hellerstein, SIGMOD 2015), with the Hydro simulator standing in for Dedalus. The important difference is who writes the correctness property. Molly's user wrote it by hand. For eventual consistency, hydrofrac's user writes nothing: a Hydro programmer who labels a cluster collection `EventualConsistency` has already stated that every member of the cluster ends up holding the same contents, and hydrofrac takes that statement as the specification.

This document describes what exists today. The development record, with the reasoning behind each decision, is `history-hydrofrac-development.md`.

Why hydrofrac is needed

A collection living at a Hydro cluster carries one of two consistency levels in its location type, `Cluster<'a, Tag, Con>`. The default, `NoConsistency`, says members may hold arbitrarily different values. `EventualConsistency` says that every member of the cluster that is given the same inputs ends up holding the same contents. A collection gets the stronger label in one of three ways:

- a transport promises it (`broadcast_closed` over a transport whose `ConsistencyGuarantee` is `EventualConsistency`);
- a programmer asserts it, with a proof (`assert_has_consistency_of(manual_proof!(...))`);
- an operator whose algebra preserves it is applied to a labelled input (a `fold` with a commutative, idempotent combiner; a `map`; a `filter`).

Collections at a single `Process` carry no label, because there is only one holder and nothing to disagree with.

The type system checks this label over the *shape* of every execution: given the same inputs, the operators between the inputs and the collection produce the same contents at every member. What the type system cannot see is whether every member actually IS given the same inputs. In a real execution a sender may crash between two sends of one fan-out, or a message on a lossy channel may be dropped. In those executions the label's premise ("given the same inputs") is false, and the label says nothing. A `broadcast_closed` over a `fail_stop` transport carries the label, and it is also true that if the sender dies halfway through its fan-out, member 0 has the item and member 1 does not.

So the label covers half of eventual consistency: agreement, given equal inputs. Hydrofrac covers the other half: whether the program, under the faults its own transports admit, actually delivers equal inputs to every correct member. Together they are the complete property. Hydrofrac does this by taking one concrete execution (a fixed topology and input workload, chosen by the test) and searching exhaustively over fault sets within

declared bounds for one that leaves a surviving member missing something another survivor has. If it finds none, the certificate says that in every explored run, every correct member held every element of the labelled collection. If it finds one, it names the fault and the schedule.

The division of labour is worth stating plainly. The type system quantifies over all executions and speaks to shape. Hydrofrac quantifies over all faults within bounds and speaks to one execution. A certificate is a statement about that execution; confidence that it generalizes to other inputs and larger clusters is the kind of confidence a test gives, sharpened by the fact that the search over faults is exhaustive within its bounds rather than random. A counterexample, on the other hand, is a statement about the program.

Working backwards from a good outcome

Hydrofrac is an LDFI tool, and it helps to recall what that means. Ordinary fault injection works forwards: pick some faults, run the system, see what happens. The space of possible fault sets is enormous and most of them are harmless, so this is a slow way to find the ones that matter.

LDFI works backwards. Start with a run in which nothing goes wrong and the outcome is good. Ask: what had to go right for this outcome? The answer is the outcome's *lineage*: the particular messages whose delivery the outcome depended on. Any fault set that leaves every one of those deliveries intact cannot change the outcome, so there is no point trying it. The only fault sets worth trying are those that kill at least one delivery in the lineage.

Now try one of those. Two things can happen. Either the outcome breaks, and we have a counterexample. Or the outcome survives anyway, which means the program had another way to produce it (a retransmission, a redundant path, an echo from a peer), and the new run's lineage shows what that other way was. That lineage becomes a second constraint: a breaking fault set must now kill something in the first lineage AND something in the second. Keep going. Each successful run teaches the search one more way the program can succeed, and the set of candidate fault sets shrinks. The search ends when no fault set within bounds can hit every lineage seen so far (a certificate), or when one of them breaks the program (a counterexample).

That is the entire idea. What hydrofrac adds to it is a Hydro-specific fault model, Hydro-specific lineage, and a specification that comes from the program's own types rather than from the user.

What hydrofrac checks, exactly

The specification is generated from the label (`Spec::eventual_consistency`). In words:

1. Run the program once with no faults. Write down every element that ends up in the labelled collection `C` at any member of the cluster. Call that set `FACTS`. It is what the program delivers when nothing goes wrong.
2. Run it again with faults. Ignore the members that crashed. The survivors must agree: no element of `FACTS` may be held by one survivor and missing from another.

That is all. The restriction to `FACTS` is what "should have been delivered" means. The phrase "held by one survivor" is there because a fault can keep an element from entering the system at all (the only sender died before sending it); then nobody has it, and that is not disagreement. The property is that survivors converge on the same set, not that the set is complete.

Two boundary cases. If the failure-free run delivers nothing (`FACTS` is empty) there is nothing to check, and the search reports `Vacuous` rather than a certificate. If a particular fault set leaves the survivors holding nothing at all, the specification is trivially satisfied and that run contributes nothing; it counts neither for nor against the

program.

A certificate is bounded in four ways, and the bounds are printed with it.

- **The fault model is the one the program declares.** Every `Network` operator is a fault point, and the transport it names is its vocabulary. A `lossy_delayed_forever` channel admits omission of any message sent before the end of finite failures. A `fail_stop` channel admits only its sender's crash. Crashes are available for the nodes the test registers as crashable, and are counted against a crash budget.
- **The bounds are the test's.** The crash budget; the end of finite failures (`EFF`), after which no new fault takes effect and the protocol has the rest of the run to converge; and the horizon, which is however long the test body drives the run.
- **The execution is the test's.** Cluster sizes, inputs, timer firings.
- **The schedules are the simulator's.** Under `Exploration::Exhaustive`, every batching and ordering the simulator can produce is explored for every hypothesis, and the certificate is a proof within the bounds. Under `Exploration::Fuzz { instances }`, a sample of schedules is explored, and the result is reported as a fuzz result, not a certificate.

Why the certificate is sound

The backwards argument of the previous section has a hidden requirement. "A fault set that leaves every delivery in the lineage intact cannot falsify the specification" is only true if the specification is *monotone* in deliveries: adding deliveries can never make a true specification false. "Every correct member holds every fact in a fixed set `FACTS`" is monotone by construction, which is exactly why `FACTS` is fixed from the probe run rather than quantified over whatever each run happens to produce. The tempting alternative, "every member holds everything any member holds," is NOT monotone over an open domain: a fault that causes one member to derive a fresh fact (a timeout firing, say) falsifies it without removing anything from any derivation, so lineage could never point the search at it. Checking that property soundly would need a syntactic analysis of the view. Hydrofrac has no such analysis, and the generated specification does not need one.

Specifications the user writes by hand (see below) do not get this guarantee. Their certificates are printed as **RELATIVE**: sound within the fault model, relative to the form of the specification the user wrote.

What a certificate is not

It is not an agreement claim: agreement given equal inputs is the type system's, and hydrofrac discharges the premise without re-proving the conclusion. It is not a liveness claim: the run has a finite horizon and the specification is evaluated at its end. A protocol that would have converged given more time is reported as violated under a body that gave it less; this says as much about the body as about the protocol, and the counterexample shows what was still missing so the author can tell which. It is not a claim about inputs or topologies other than the test's. And hydrofrac chooses fault sets, not interleavings; interleavings are the simulator's.

Using hydrofrac

Nothing is required of the program under test. The test fixes an execution, names the labelled collection, and drives the run:

```
let mut flow = FlowBuilder::new();
let source = flow.process::<Source>();
let members = flow.cluster::<Member>();
```

```
let (send, items) = source.sim_input::<u32, TotalOrder, ExactlyOnce>();

let delivered = reliable_broadcast_closed(items, &members);   // labelled EventualConsistency

let mut spec = Spec::new(&mut flow);
let correct = spec.correct_members(&members, 0..3);
let observed = spec.observe_cluster(delivered.weaken_consistency());
spec.eventual_consistency(observed, correct);

let compiled = flow.sim()
    .skip_consistency_assertions()      // the simulator does not yet check them
    .with_cluster_size(&members, 3)
    .with_provenance()
    .compiled();

Ldfi::new()
    .level(Level::Record)
    .crashable("source", &source)
    .crashable_cluster("member", &members, 0..3)
    .crash_budget(1)
    .spec(&spec)
    .search(&compiled, async |_run| {
        send.send(1);
        send.send(2);
        hydro_lang::sim::quiesce().await;
    })
```

The body is the execution: which inputs, which timers, how long. For a protocol whose convergence needs time (retransmission timers, gossip rounds, leader change), the body fires those timers after `run.end_of_finite_failures()` so the protocol gets its window. The result prints either as a counterexample, with its faults and the failing schedule, or as a certificate, with its grade, hypothesis count and exploration mode.

Two variants exist for comparison. `Strategy::Blind` runs the same loop over the same fault universe without lineage (hypotheses are simply the untried fault sets in order of increasing weight); it is the ablation that measures what lineage buys. `Ldfi::fuzz_faults_with` samples fault sets at random.

How hydrofrac works

Fault points

The Hydro simulator compiles a flow into a single-process executable in which every network channel is a queue, every tick is an explicitly scheduled batch, and every external input and timer is driven by the test body. It can explore batchings and orderings exhaustively or by sampling. Hydrofrac places a filter in front of every network send. The filter knows two verbs:

- **Omission.** Drop this message, identified by channel, sender, destination and wire bytes. Every byte-identical copy on that channel is dropped while finite failures last, so a retransmission with the same payload counts as the same message. Available on lossy channels.
- **Crash.** `crash(node after m)`: the node performs every send up to and including `m`; every later send is dropped and its external outputs are suppressed. The crash is named by a message rather than a send count so that it means the same thing in every schedule, and so that it can split the fan-out of a single broadcast (the sender dies between two sends of the same item). Available on nodes the test registers.

Both kinds of fault lie before `EFF`. The program registers nothing; the transports it already declares are the fault model.

Lineage

Every run records every execution (an async run, or one execution of a tick), every send with its fate, every receive with the delivery that satisfied it, and which execution produced each output. The *derivation* of a specification's view is the backward closure from the executions that produced it. Four levels of precision are available:

Level	What a tick execution is taken to depend on
`Exec`	every input of this execution and of every earlier execution of the same tick
`Slice` (default)	as above, narrowed by a static slice of the program per sink
`Record`	record-level provenance, by an IR rewrite that threads a provenance set through every operator (`SimFlow::with_provenance`)
`Counterfactual`	counterfactual replay: each node is re-executed alone with one delivery removed at a time; a delivery is in the derivation iff the node stops producing the output without it

These trade precision for applicability. `Exec` and `Slice` always apply and may over-approximate: they can name deliveries the output did not actually need. `Record` and `Counterfactual` are more precise but can fail on a node. The provenance rewrite does not see through every closure, and a replay with every delivery kept sometimes does not reproduce the node's output, in which case removing deliveries one at a time proves nothing. When that happens, the derivation for that node is taken from the next less precise level: `Counterfactual` falls back to `Record`'s answer, `Record` to `Slice`'s.

The direction of the fallback matters. An over-approximated derivation makes a clause too large, so the search tries fault sets it did not need to; that costs hypotheses, not correctness. An under-approximated derivation makes a clause too small, and could hide the one fault set that breaks the program behind a certificate. Falling back toward the levels that can only over-approximate keeps every certificate sound.

The search

This is Molly's loop. The failure-free run (the probe) fixes `FACTS` and yields the first derivation. Each derivation becomes a clause: a set of deliveries, at least one of which any breaking fault set must kill, where omission kills a message and `crash(node after m)` kills every later send of that node. A hypothesis is a hitting set of all clauses so far, within the crash budget. Each hypothesis is run under the configured exploration. If the specification fails in some schedule, that is the counterexample. If it holds, the run's derivation is a new clause, which is the search learning about the retransmission, the redundant path or the echo that saved the outcome. The search ends when no hitting set exists within the bounds (a certificate), when a counterexample is found, or when the hypothesis budget runs out (`Outcome::Budget`).

The generated specification

`Spec` is itself a Hydro flow, placed at a dedicated observer process and compiled together with the program. The test names the collection the label is on (`spec.observe_cluster(collection)`) and the cluster's members (`spec.correct_members(&cluster, 0..n)`). Before each run, hydrofrac feeds the observer the members the hypothesis does not crash and the `FACTS` learned from the probe. `spec.eventual_consistency(observed, correct)` builds the view: the distinct `(member, fact)` pairs at correct members over `FACTS` must number as many as the `FACTS` facts present times the number of correct members. The view is evaluated at the end of each run.

Because the observer is part of the compiled flow, the specification's derivation is recorded like any other output's, and the same lineage machinery produces the clauses.

Which collections are checked

Today a test checks one labelled collection, the one it passes to `spec.observe_cluster`. A program may label several (multi-Paxos labels thirteen), and a test says nothing about the ones it does not name. Every search therefore begins by listing every place in the program where a collection acquires the label, marked `checked by this test`, `UNCHECKED (fed by a network channel)`, or `nothing to check (local inputs only)`, so the scope of a certificate is visible in the log.

This is interim. Since the specification is a function of the collection alone, and the labelled collections are known from the program's IR, nothing about this step actually needs the test author. The next version observes every labelled, network-fed cluster collection automatically (by an IR rewrite at flow finalization, the same way the provenance rewrite already inserts operators) and runs one search per collection against the shared test body, reporting a verdict for each: certified, violated, or not in class with the reason (a lattice `Singleton`, a collection inside a tick). The report above then goes away; `spec.observe_cluster` remains for checking one collection by hand.

Sorting the existing code: certified or incorrect

Every producer of the `EventualConsistency` label in the codebase, and every demonstration protocol in `hydro_std::ec_inference_demos` whose labelled output is a cluster-resident set, has been run under the generated specification with one crash allowed and every registered node crashable (`hydro_test/src/cluster/ec_ldfi.rs`, `ec_demos_ldfi.rs`). Each falls into one of three bins. Hypothesis counts include the probe run.

Incorrect: the label is violated by one fault.

Program	Result
`broadcast_closed`, process → cluster, `fail_stop`	**Violated**, 5 hypotheses: `crash(source after bcast[→1]:Item(2))`. The sender dies between two sends of one item.
`broadcast_closed`, process → cluster, `lossy_delayed_forever`	**Violated**, 2 hypotheses: `omit(bcast[→0]:Item(2))`. One lost message; nothing retransmits.
`broadcast_closed`, cluster → cluster, `fail_stop`	**Violated**, 5 hypotheses: `crash(source@0 after bcast[0→1]:Item(10))`.
`broadcast_live_from_process` (one hop over live membership)	**Violated**, 5 hypotheses: `crash(source after bcast[→1]:2)`.
`leader_merge_broadcast`	**Violated**, 2 hypotheses: `crash(leader after merged[→1]:(writer 0, 100))`. The demo's own documentation says its label assumes the leader survives; this is that sentence as a counterexample.

Certified: no fault set within bounds breaks the label.

Program	Result
Echo reliable broadcast (`hydro_test` version)	**Certificate**, 10 hypotheses, exhaustive.
`reliable_broadcast_closed`, `reliable_broadcast_live`	**Certificate**, 10 each, exhaustive.
`uniform_reliable_broadcast_closed`, thresholds 1 and 2	**Certificate**, 28 each, 8 sampled schedules per hypothesis.

Program	Result
`g_set_gossip`, `g_set_gossip_live`	**Certificate**, 13 each, 8 sampled schedules.
`multi_paxos`, the learners' `learned` output (1 proposer, 3 acceptors, 3 learners)	**Certificate**, 24 hypotheses, 8 sampled schedules. The learners re-broadcast what they learn; the certificate belongs to that echo cycle. The proposer is held correct.

The pattern is clean. Every single-hop dissemination fails under one crash or one omission; every protocol with an echo or gossip cycle holds. The transports themselves (`networking/mod.rs`) justify granting the label by reasoning about the recipient ("a failed connection models the recipient as failed") or about delay ("dropped messages are indefinitely delayed"), and neither argument survives one sender crash or one lost message. The label is really a promise about the program that uses the transport, and only a program that retransmits keeps it. This is the right division of labour: the type system cannot know whether the program retransmits, and hydrofrac can.

Not a fault question, or not in class. The remaining labelled collections in the tree do not need hydrofrac, or cannot yet use it. `source_interval` on a cluster is a stream of `()`. Keyed folds with algebraic proofs preserve the label given the same inputs, which is exactly what the type system already checks. `collect_quorum` is a function of its response set. Of the demos, `quorum` and `epoch_splice` are combinators with no transport of their own (EC in, EC out; their premise is whichever broadcast fed them), `synod` labels nothing `EventualConsistency`, and `abd`'s replica register is in the class but is a lattice `Singleton`, which the set-based specification cannot observe correctly (see Limitations).

Hand-written specifications

A `Spec` can also be written by hand, as two Hydro views over the program's outputs: a pre-condition (when it is empty the run is ignored) and a post-condition (when it is empty and the pre-condition is not, the run is a counterexample). `Ldfi` also accepts simpler goals over plain outputs (`goal`, `goal_all_members`, `goal_witnessed`). This is how hydrofrac is run against properties the type system does not state: Raft's and Paxos's durability of committed entries, two-phase commit's atomicity, ISR replication's agreement, quorum's claim counting, retry-until-ack delivery (`raft_ldfi.rs`, `paxos_ldfi.rs`, `two_pc_ldfi.rs`, `isr_replication.rs`, `quorum_ldfi.rs`, `reliable_delivery.rs`). Their certificates are graded RELATIVE, since no form argument grades them, and the user is told so.

They have found real bugs. An injected Raft election-restriction bug (comparing logs by index only) is found in 90 hypotheses with four omissions and no crash, a scenario the author had not written down; correct Raft certifies in 235 hypotheses under the same bounds. (Both tests are `#[ignore]`d for running time: thirty minutes and an hour.) Paxos's P1b quorum counted replies rather than acceptors, a defect in the production protocol that surfaced once the body's `EFF` was moved past the first ballot; correct Paxos, the Paxos key-value store and compartmentalized Paxos certify in 372, 301 and 568 hypotheses (ignored, 45 minutes together). The simulator also turned up two defects along the way that are not fault questions: `collect_quorum_with_response` was not a function of its input (its `nondet!` justification was false; fixed, with a test), and a `fold` with an idempotence proof had no simulator implementation of its retry-only cast (now the identity, since the simulated network never duplicates).

Limitations

- **One execution.** The certificate is for the test's topology and inputs. Nothing in hydrofrac generalizes it; that induction is the test author's.

- **Sampled schedules are not certificates.** Protocols with large schedule spaces (uniform broadcast, gossip, multi-Paxos) run under `Fuzz` and are reported as such.
- **The horizon is the body's.** A protocol gets the time the body gives it. An unconverged run is a counterexample, not vacuous. A convergence witness reported by the protocol itself (every correct member has quiesced on the same view) would make such runs vacuous instead; it is not built. Raft's hand-written pre-condition ("some member committed") is where this cost effort: its body was tuned until correct Raft converged.
- **Set-valued collections only.** The generated specification observes a stream of facts. A lattice `Singleton` (`abd`'s max-by-timestamp register) needs a variant that observes the final value per member and checks equality; not built.
- **Static membership.** The simulator has everyone joined from the start; the `_live` demos are tested for faults, not for join timing.
- **Lineage gaps.** The provenance rewrite does not handle `entries_partially_ordered` on a keyed stream (`leader_merge` runs at `Level::Slice`). Counterfactual replay can panic inside a program that assumes a delivery it is being denied; the panic is contained on the generated side of the dynamic-library boundary and treated as inconclusive for that node.
- **The report of unchecked labels is only a report.** It does not fail a test. Making it fail one needs a notion of "every search over this program", which a per-program registry of specifications could provide.
- **Byzantine, partition and duplication faults** are outside the vocabulary. Partitions are expressible as omission sets on lossy channels and are found that way (the Raft counterexample is four omissions on two links); duplication has no fault point.

Next

In the order the argument of this document needs them:

1. Automatic coverage of every labelled collection, in progress.
2. The lattice-aware specification for `Singleton` outputs, and `abd` under it.
3. A convergence witness in the generated specification for protocols that report one, so that horizon becomes a parameter rather than a verdict.
4. Decision-trace replay: record every nondeterministic decision of a run and replay a prefix with one decision overridden. This is the general counterfactual oracle of which node-level replay is the coarse instance.
5. Raft's `committed` under the generated specification. The harness is written; the run is an hour.